

Documento de Arquitectura y Decisiones Técnicas

Justificación formal de las decisiones tecnológicas adoptadas: lenguaje, framework, motor de base de datos, infraestructura cloud y servidor web.

Versión 1.1 · Junio 2026

Introducción

Este documento expone los fundamentos técnicos que sustentan la elección del stack tecnológico de IMSGa, abarcando el desarrollo móvil, el backend, la infraestructura cloud, el motor de base de datos, el sistema operativo de los servidores y el servidor web. Cada decisión se evaluó bajo un mismo criterio: mantener la simplicidad del desarrollo sin sacrificar estabilidad, rendimiento ni capacidad de escalar a futuro.

Aplicación Móvil

React Native

Framework de desarrollo móvil basado en React, seleccionado por su arquitectura orientada a componentes y su compatibilidad con las características nativas de Android. Esta combinación permite una navegación fluida dentro de la aplicación, reutilizando elementos de interfaz ya existentes en lugar de construir nuevos componentes desde cero.

TypeScript

Superset de JavaScript que incorpora tipado estático: cada variable declara explícitamente el tipo de dato que contendrá, y ese tipo se mantiene fijo durante la ejecución del programa. Esto contrasta con el tipado dinámico nativo de JavaScript, donde el tipo de una variable se infiere de su valor inicial pero puede cambiar libremente en tiempo de ejecución —por ejemplo, una variable inicializada con un string puede luego recibir un valor numérico sin que el lenguaje lo impida—. El tipado estático de TypeScript reduce esta clase de errores y aporta una estructura de datos más predecible.

Backend

Lenguaje: Python

Se eligió Python por permitir un desarrollo ágil con bajo consumo de recursos. Es un lenguaje altamente portable, que no requiere instalar numerosas dependencias adicionales para ejecutarse en distintos servidores, es fácil de mantener en el tiempo y cuenta con documentación extensa, lo que se ajusta al objetivo central del proyecto: mantener la simplicidad del desarrollo.

Framework: Django

Django permite administrar la mayor parte de la lógica del backend desde un único lugar, gracias a un ORM robusto que gestiona la creación de tablas, índices y relaciones directamente desde el código, sin necesidad de mantener scripts de base de datos en paralelo. Esto no reemplaza la posibilidad de optimizar el esquema manualmente en el futuro, pero sí simplifica el trabajo más rutinario durante el desarrollo. Adicionalmente, Django ofrece compatibilidad prácticamente nativa

con buena parte de los servicios de AWS utilizados por el sistema, y cuenta con documentación extensa que reduce la curva de aprendizaje del equipo.

Procesamiento Asíncrono: Celery y Redis

Las tareas de larga duración —como el envío de notificaciones push y el registro de logs de auditoría— se desacoplan del ciclo de solicitud/respuesta mediante Celery, utilizando Redis como broker de mensajes. Redis y el worker de Celery se ejecutan sobre una instancia EC2 dedicada, separada del backend. Redis se eligió por su bajo overhead y su compatibilidad directa con Django, evitando introducir un servicio gestionado adicional más complejo de mantener. El worker se ejecuta con un pool gevent, lo que resuelve las incompatibilidades de monkey-patching entre Celery y Python 3.14.

Infraestructura Cloud

AWS

Se eligió AWS como proveedor de infraestructura cloud por su compatibilidad prácticamente nativa con Django, lo que permite integrar los servicios necesarios sin invertir tiempo adicional de desarrollo en adaptadores o configuraciones complejas. Su facilidad de despliegue y su documentación extensa reforzaron esta decisión, alineada con el criterio de simplicidad que guía las demás elecciones tecnológicas del proyecto.

AWS Secrets Manager

Las credenciales del sistema (clave de Django, contraseña de base de datos, llaves de acceso a AWS) se gestionan a través de AWS Secrets Manager, en reemplazo del enfoque inicial basado en un archivo de variables de entorno almacenado en texto plano en el servidor. Este cambio elimina la exposición de secretos en el sistema de archivos y fuera del control de versiones, permite la rotación centralizada de credenciales sin intervención manual en cada instancia, y condiciona el acceso a roles IAM en lugar de depender de quién tenga acceso por SSH al servidor.

Motor de Base de Datos

PostgreSQL (elegido)

PostgreSQL fue seleccionado como motor de base de datos por las siguientes razones:

- Alto rendimiento y velocidad de respuesta.
- Sin costo de licenciamiento.
- Compatibilidad prácticamente nativa con Django, a excepción de SQLite.
- Estabilidad comprobada en entornos de producción.
- Soporte de características avanzadas, como el tipo de dato JSONB.
- Facilidad de migración y escalabilidad del esquema.

Su integración con Redis permite además mejorar la velocidad de respuesta en consultas frecuentes mediante cacheo, sin comprometer la simplicidad del desarrollo.

Alternativas descartadas

Alternativa	Motivo de descarte
Oracle	Requiere configuración previa compleja (wallets para conexiones desde Java) y conlleva costos de licenciamiento, lo que implicaría una pérdida significativa de tiempo de desarrollo solo en la fase de integración.
Bases de datos NoSQL	No cuentan con integración nativa en Django. Su principal ventaja —alto rendimiento ante consultas masivas y concurrentes— no resulta determinante para el volumen de uso proyectado del sistema, donde PostgreSQL ya garantiza un rendimiento adecuado.
MySQL / MariaDB	Priorizan la velocidad de escritura por sobre la integridad de los datos, un trade-off inaceptable en un sistema que maneja información clínica crítica. A diferencia de PostgreSQL, no soportan transacciones en cambios de esquema: si una migración falla a mitad de camino, la base de datos puede quedar en un estado corrupto e inconsistente. Además, requieren configuración adicional de collations (como utf8mb4) para soportar correctamente caracteres especiales y emojis, algo que en PostgreSQL es nativo.

Sistema Operativo

Los servidores del sistema utilizan Linux por su bajo costo operativo, facilidad de escalamiento y portabilidad entre proveedores. A diferencia de sistemas operativos como Windows o macOS, Linux permite un control granular sobre la configuración del servidor, facilitando el diagnóstico de fallos y la personalización del entorno según las necesidades del proyecto, además de ofrecer un mejor rendimiento general para cargas de trabajo de servidor.

Servidor Web

NGINX fue elegido como servidor web por su compatibilidad prácticamente nativa con Django, su capacidad de gestionar certificados SSL de forma automática mediante Let's Encrypt, y su flexibilidad para configurar proxys reversos, reglas de seguridad y protocolos de comunicación. Estas características permiten mantener y escalar la infraestructura del sistema con una inversión de tiempo reducida.

Control de Versiones y CI/CD

Se eligió GitHub como plataforma de control de versiones y, junto con GitHub Actions, como motor de integración y despliegue continuo (CI/CD) del sistema. Esta decisión responde al mismo criterio de simplicidad que guía el resto de las elecciones tecnológicas del proyecto:

- Plataforma única para código y CI/CD: al alojar el repositorio en GitHub, los flujos de despliegue se ejecutan sobre la misma plataforma, sin necesidad de levantar ni mantener un

servidor de integración continua adicional (como Jenkins) ni migrar el código a otro proveedor solo para automatizar el despliegue.

- Integración nativa con eventos de git: GitHub Actions dispara automáticamente el flujo de despliegue (git fetch/reset sobre las instancias EC2 del backend y del worker de Celery) ante eventos como un push a la rama de producción, sin pasos manuales adicionales.
- Sin costo de licenciamiento: la capa gratuita de GitHub Actions cubre las necesidades de automatización del proyecto.
- Documentación extensa y adopción amplia en la comunidad, lo que reduce el tiempo de configuración y mantenimiento de los flujos de trabajo.

Módulo Criptográfico: Rust

La generación de la firma electrónica (hash SHA-256 y firma Ed25519, ver Requisitos Funcionales y No Funcionales relacionados a firma electrónica) se implementa en un módulo nativo escrito en Rust, integrado al backend de Django mediante bindings de PyO3. Se optó por Rust específicamente para este componente, en lugar de mantenerlo en Python, por las siguientes razones:

- Gestión de memoria determinista: al no depender de un recolector de basura, Rust garantiza el momento exacto en que la memoria que contiene la clave es liberada, evitando que esta persista en memoria más tiempo del necesario.
- Zeroización de la clave: tras su uso, el valor de la clave en memoria es sobrescrito explícitamente, reduciendo la ventana de exposición ante un eventual volcado de memoria.
- Aislamiento mediante OnceLock: la clave se mantiene en una única ubicación de memoria, inicializada una sola vez, lo que impide que se dupliquen copias o que sea leída desde fuera del alcance controlado del módulo.
- Rendimiento en memoria limitada: el módulo retorna directamente los bytes resultantes (hash y firma) en lugar de serializarlos como JSON, reduciendo el uso de memoria en instancias con recursos acotados.

Este mismo módulo es responsable tanto del cálculo del hash como de la generación de la firma electrónica, concentrando el manejo de la clave del sistema en un único componente auditable.